INTRO SESSION

Reinforcement Learning

Learning from trial, error, and reward

An intuitive introduction — ideas and intuition before the equations

Part 1 · Foundations

Part 2 · Deep RL

Part 3 · The Brain

REINFORCEMENT LEARNING • OVERVIEW

Where we're headed today

Three steps: the core ideas, how they scale with deep learning, and how the same logic shows up in the brain.

PART 1

Foundations

What is RL?

The Value Function

Multi-Armed Bandit

PART 2

Scaling up — Deep RL

The Q-Function

Deep RL (DQN)

Actor–Critic

PART 3

RL in the brain

The Dopamine circuit

Model-free vs. Model-based

PART 1

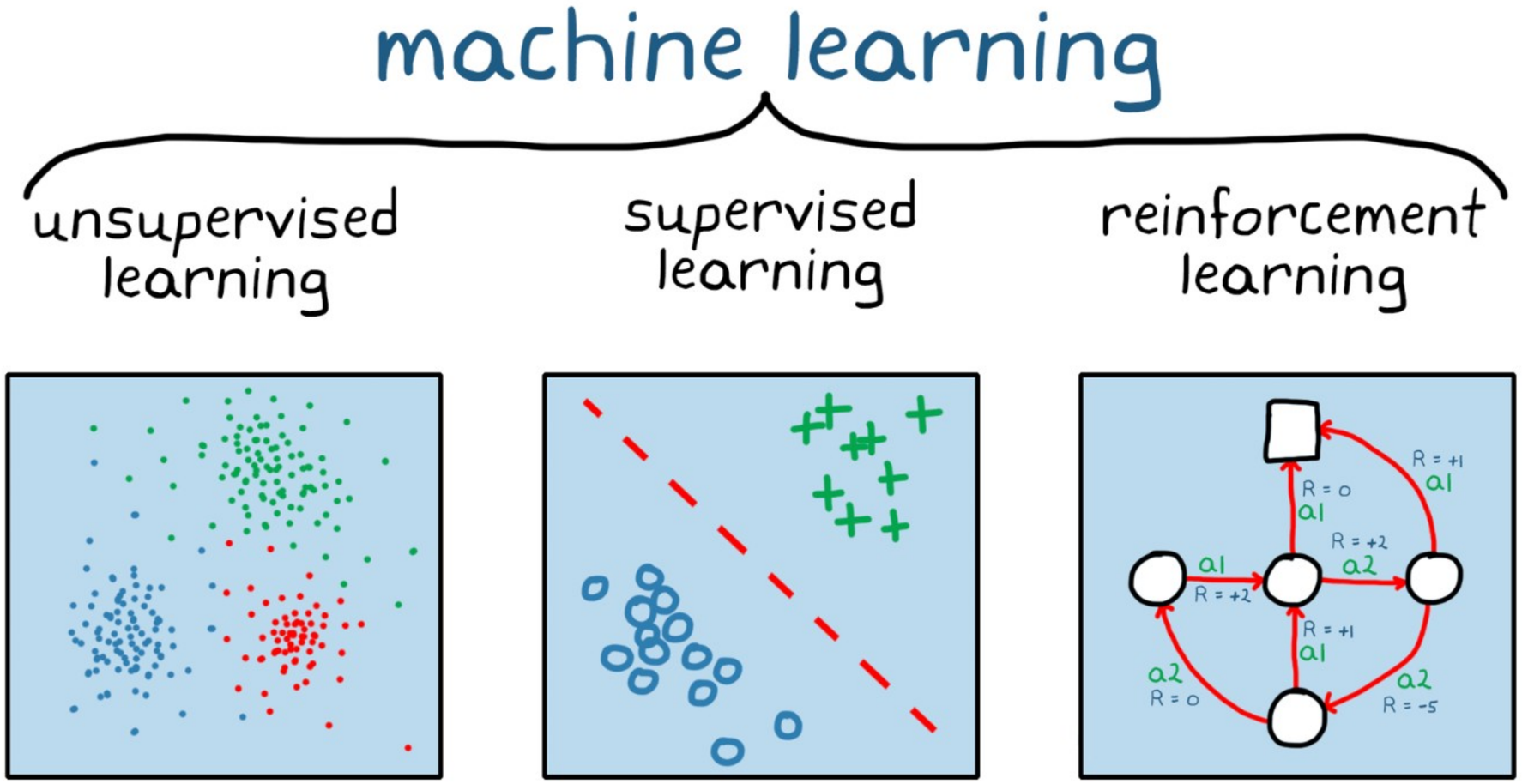
Foundations of RL

The vocabulary of Markov Decision Processes and the single core mathematical idea that everything else is built on.

1

What is Reinforcement Learning?

Machine learning



What is Reinforcement Learning?

Terminology

Agent — the learner or decision-maker that interacts with the environment

Environment — the external world or system that responds to the agent's actions

State — the current situation or condition the agent observes

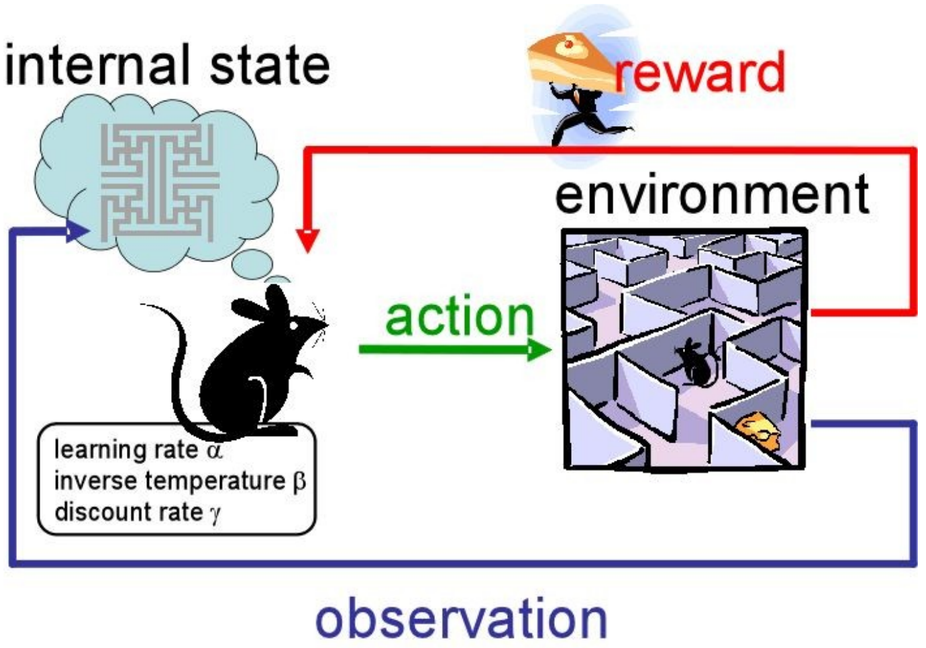
Action — a choice or move the agent can make in a given state

Reward — immediate feedback from the environment after an action

Value — the expected long-term return from a state or action

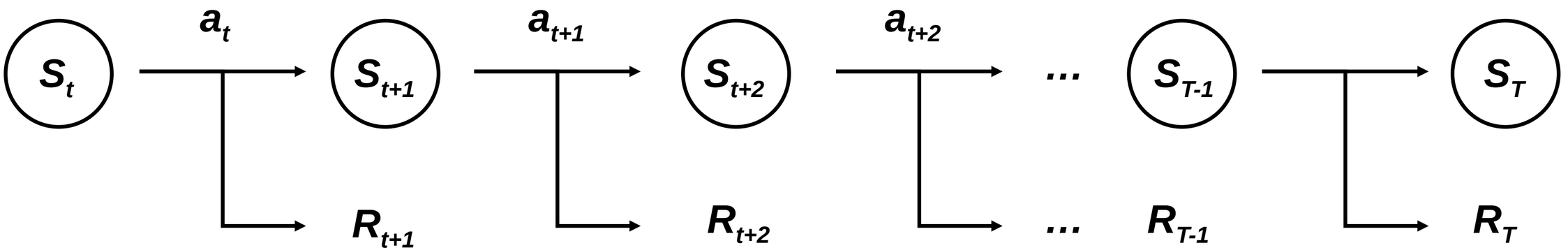
Policy (π) — the strategy: which action to take in each state

"The goal is not to maximize immediate reward, but to maximize the total long-term reward"



What is Reinforcement Learning?

Markov decision process



Markov property: The future depends only on the current state, not on the past

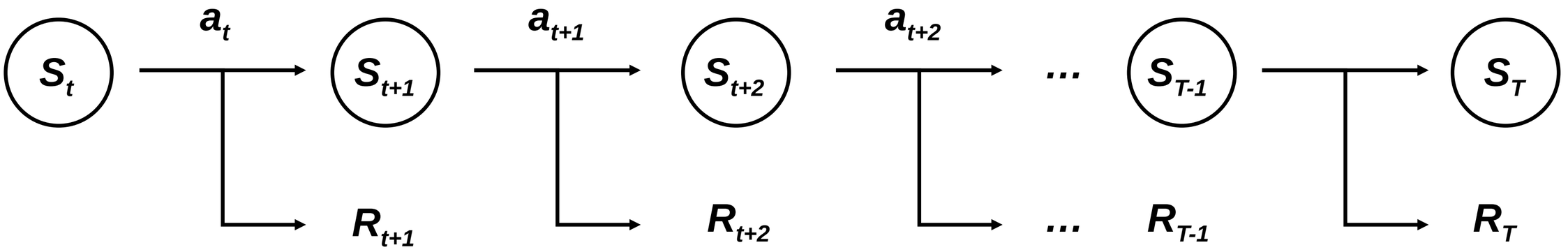
The Value Function

Value vs. Reward

Reward — what you get now

Value — what a situation is worth in the long run; the expected long-term return

Expected return



$$G_t = \gamma^0 \times R_{t+1} + \gamma^1 \times R_{t+2} + \gamma^2 \times R_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The Value Function

The Bellman Equation for $v_\pi(s)$

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_\pi(s')\right]$$

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_\pi(s')\right]$$

The probability of selecting action *a* in state *s*

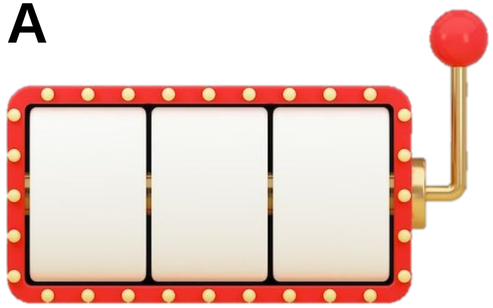
The probability of transitioning to state *s′* and receiving reward *r* after taking action *a* in state *s*.

The immediate reward plus the discounted value of the next state

The Multi-Armed Bandit

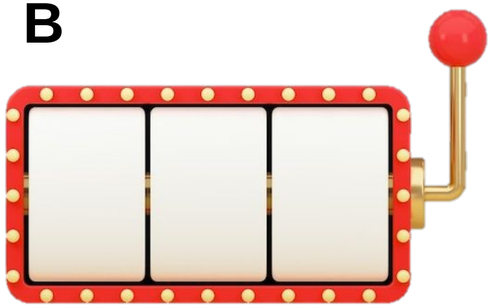
The explore vs. exploit dilemma, in its purest form

A



-100

B

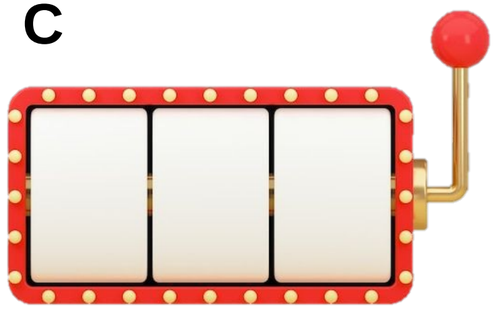


+50

+50

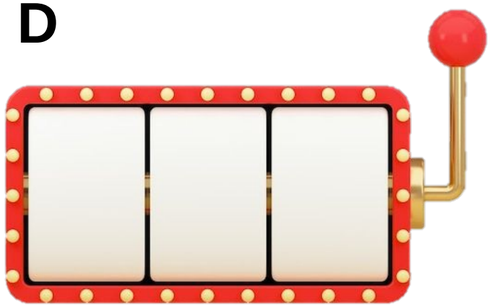
+50

C



+1000

D

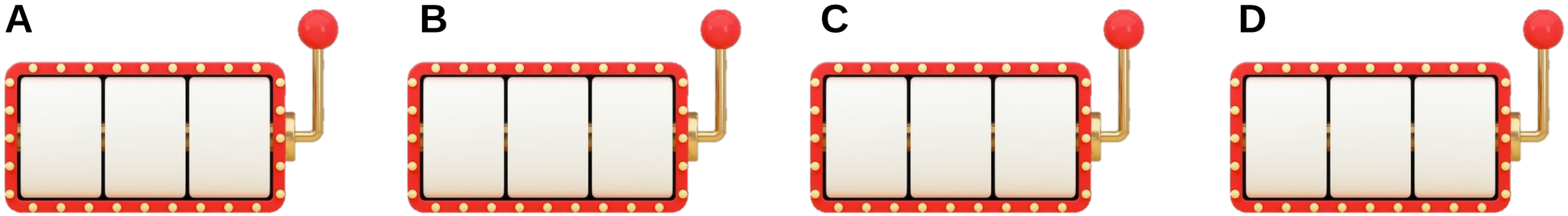


The Multi-Armed Bandit

ε-greedy

ε : probability for explore

ε = 0.1



25%

25%

25%

25%

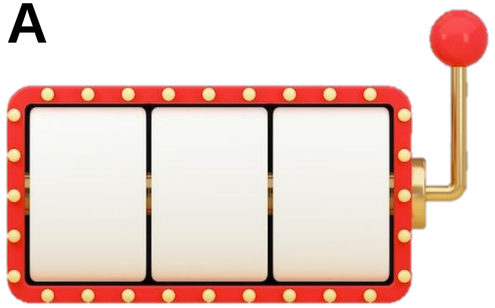
The Multi-Armed Bandit

ε-greedy

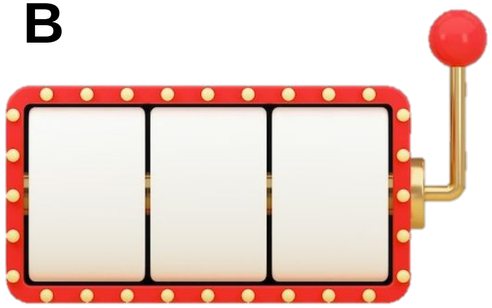
ε : probability for explore

ε = 0.1

A

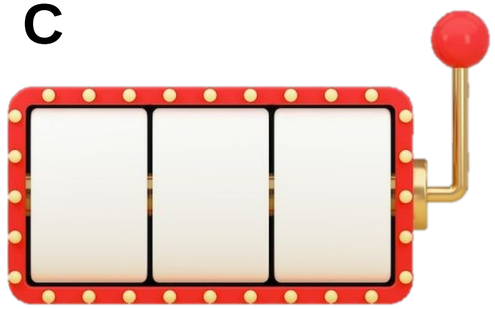


B

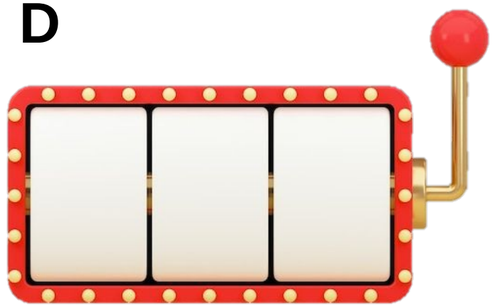


+50

C



D



25%

25%

25%

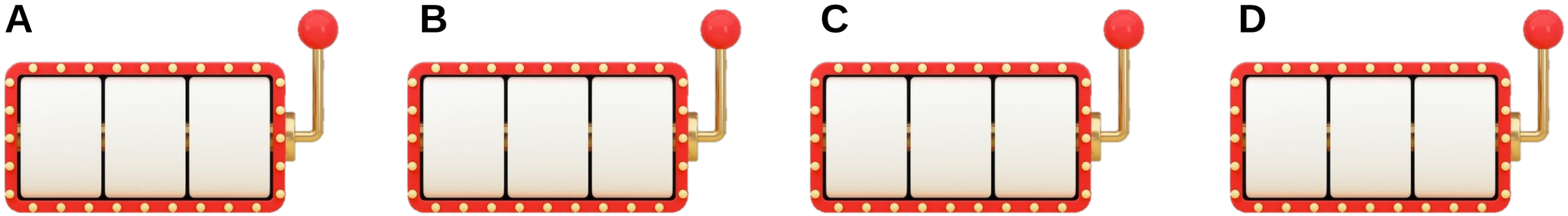
25%

The Multi-Armed Bandit

ε-greedy

ε : probability for explore

ε = 0.1



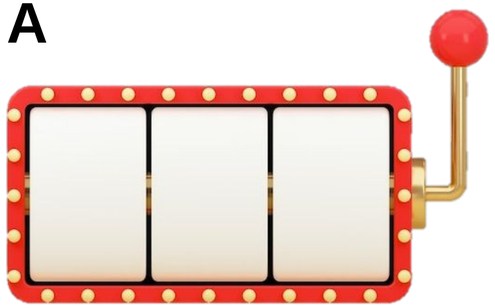
The Multi-Armed Bandit

ε-greedy

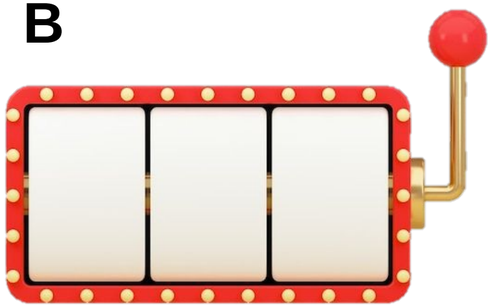
ε : probability for explore

ε = 0.1

A



B

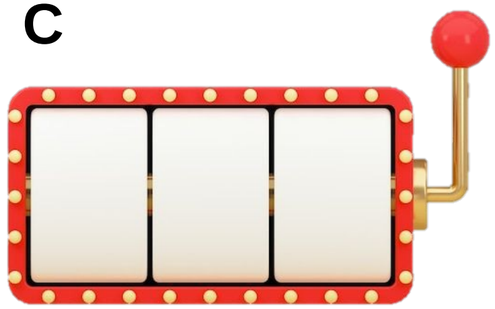


+50

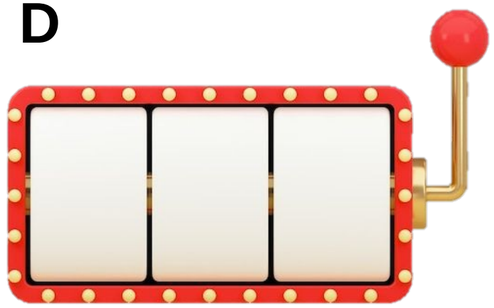
+50

+50

C



D



3.3%

90%

3.3%

3.3%

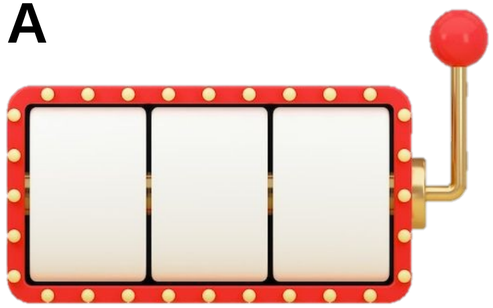
The Multi-Armed Bandit

ε-greedy

ε : probability for explore

ε = 0.1

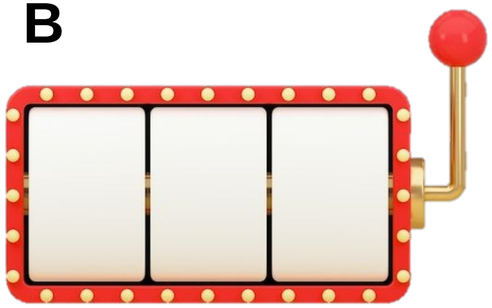
A



-100

3.3%

B



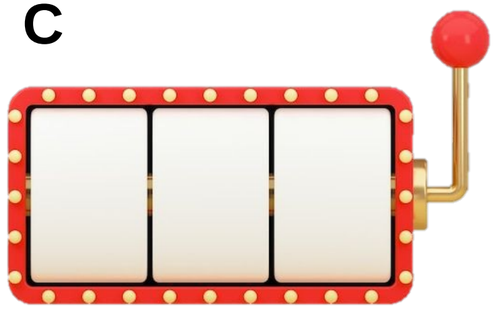
+50

+50

+50

90%

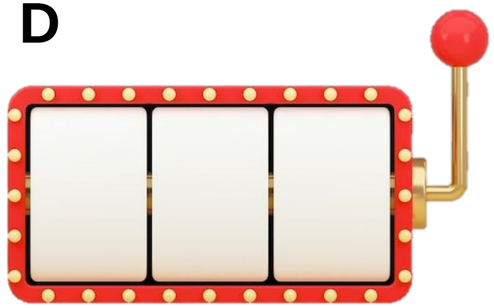
C



+1000

3.3%

D



3.3%

The Multi-Armed Bandit

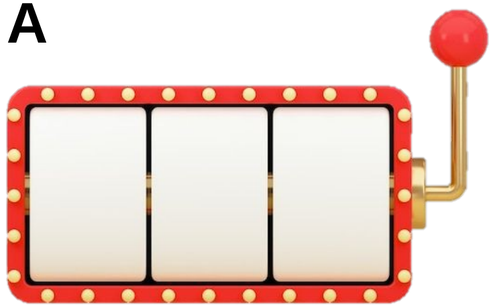
ε-greedy

ε : probability for explore

ε = 0.1

*epsilon decay: gradually decreasing epsilon

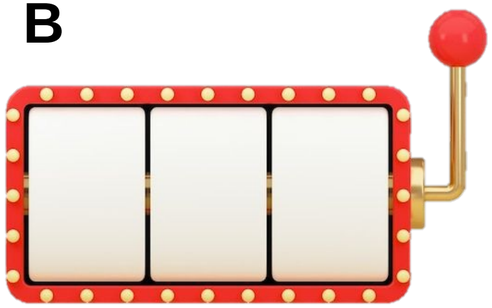
A



-100

3.3%

B



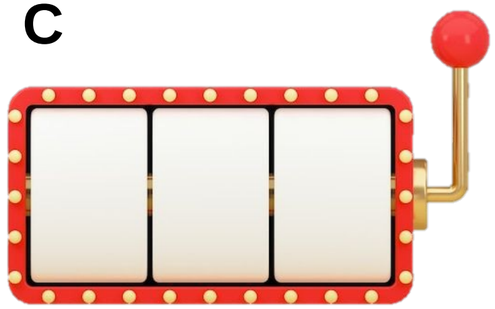
+50

+50

+50

3.3%

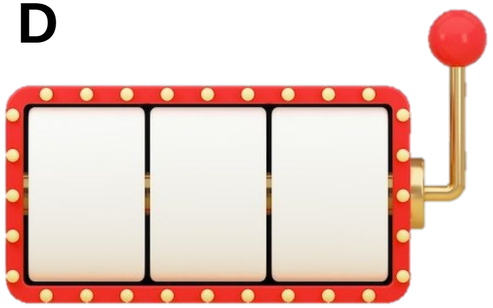
C



+1000

90%

D

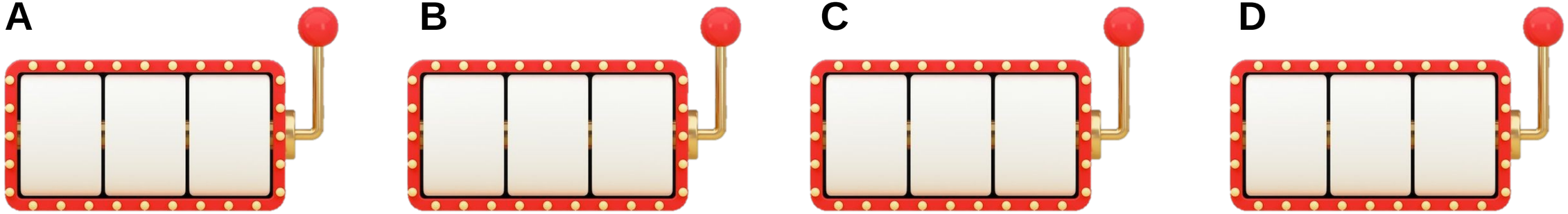


3.3%

The Multi-Armed Bandit

Softmax Action Preference

$$\pi(a|s) = e^{Q(s,a)/\tau} / \sum_b e^{Q(s,b)/\tau}$$

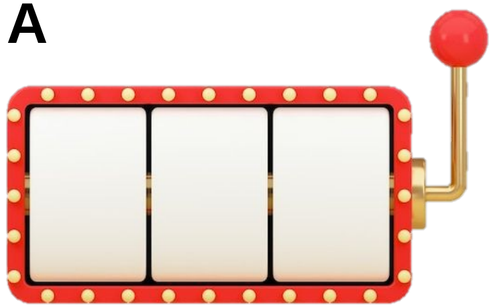


The Multi-Armed Bandit

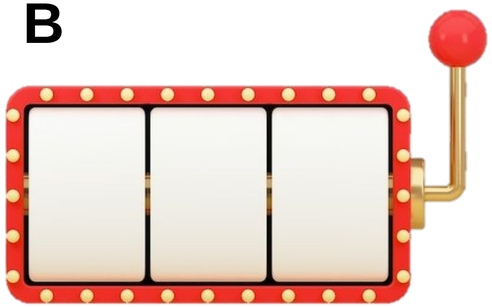
Softmax Action Preference

$$\pi(a|s) = e^{Q(s,a)/\tau} / \sum_b e^{Q(s,b)/\tau}$$

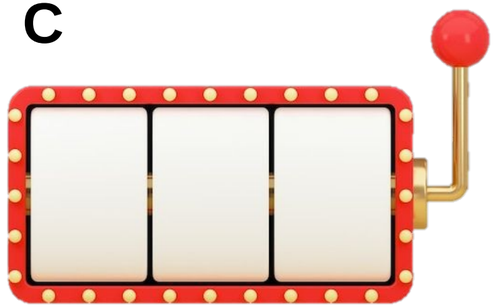
A



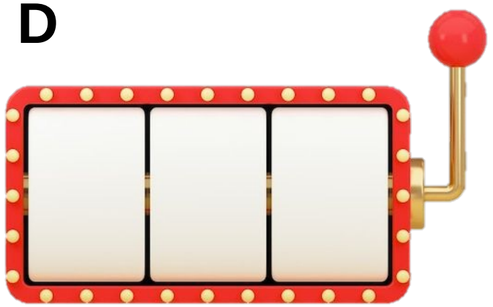
B



C



D



$$\pi = \frac{e^0}{e^0+e^0+e^0+e^0} = 25\%$$

25%

25%

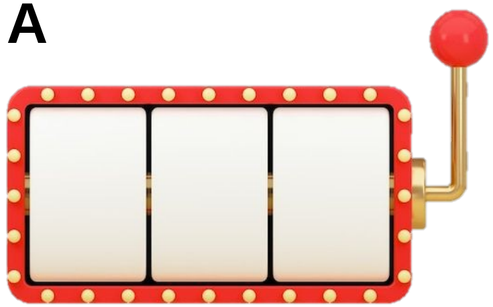
25%

The Multi-Armed Bandit

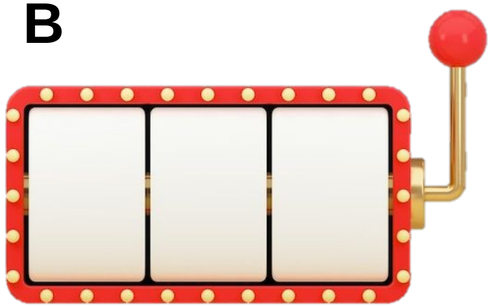
Softmax Action Preference

$$\pi(a|s) = e^{Q(s,a)/\tau} / \sum_b e^{Q(s,b)/\tau}$$

A

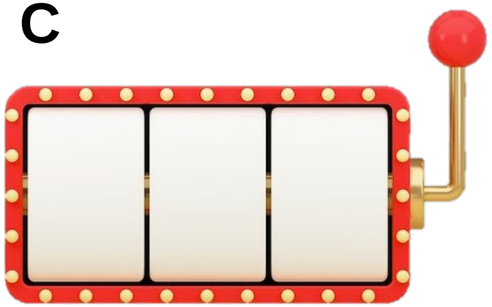


B

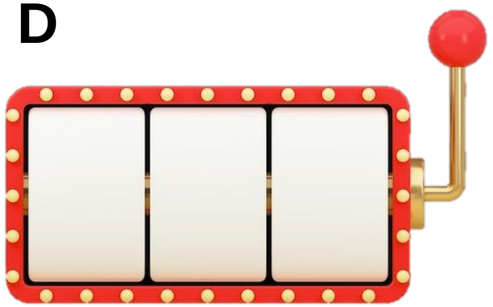


+5

C



D



$$\pi = \frac{e^0}{e^0+e^0+e^0+e^0} = 25\%$$

25%

25%

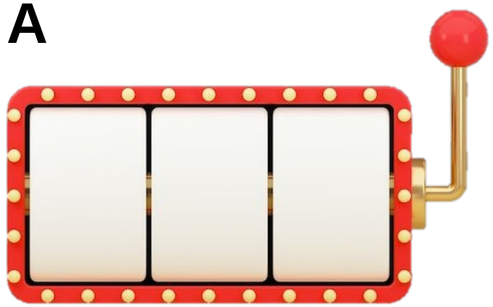
25%

The Multi-Armed Bandit

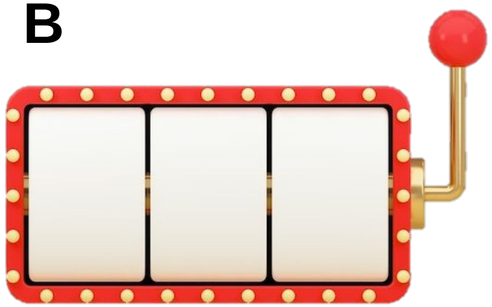
Softmax Action Preference

$$\pi(a|s) = e^{Q(s,a)/\tau} / \sum_b e^{Q(s,b)/\tau}$$

A

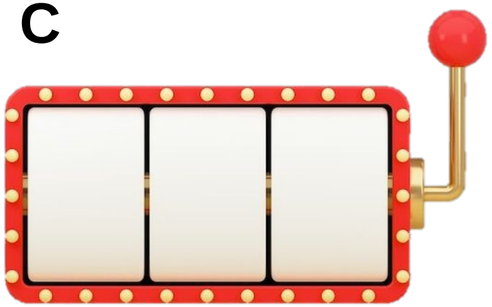


B

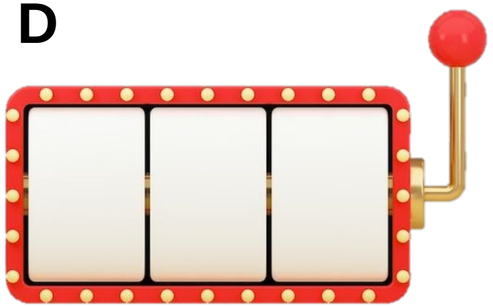


+5

C



D



$$\pi = \frac{e^0}{e^0+e^5+e^0+e^0} = 0.66\%$$

$$\pi = \frac{e^5}{e^0+e^5+e^0+e^0} = 98.02\%$$

$$\pi = \frac{e^0}{e^0+e^5+e^0+e^0} = 0.66\%$$

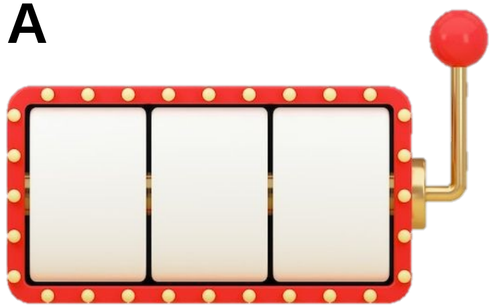
$$\pi = \frac{e^0}{e^0+e^5+e^0+e^0} = 0.66\%$$

The Multi-Armed Bandit

Softmax Action Preference

$$\pi(a|s) = e^{Q(s,a)/\tau} / \sum_b e^{Q(s,b)/\tau}$$

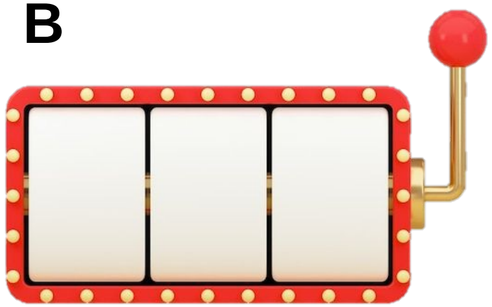
A



-3

$$\pi = \frac{e^0}{e^0+e^5+e^0+e^0} = 0.66\%$$

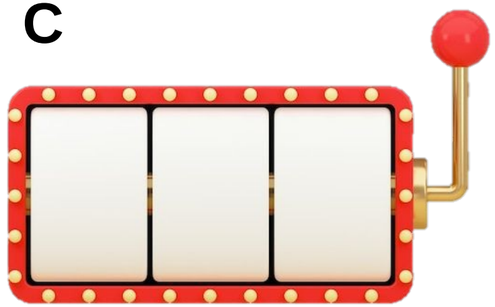
B



+5

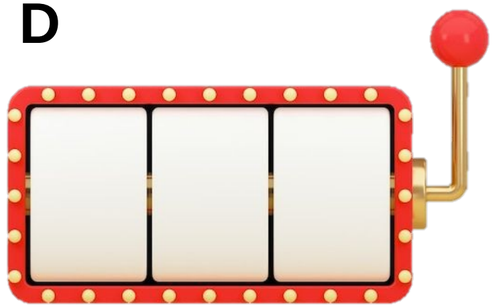
$$\pi = \frac{e^5}{e^0+e^5+e^0+e^0} = 98.02\%$$

C



$$\pi = \frac{e^0}{e^0+e^5+e^0+e^0} = 0.66\%$$

D



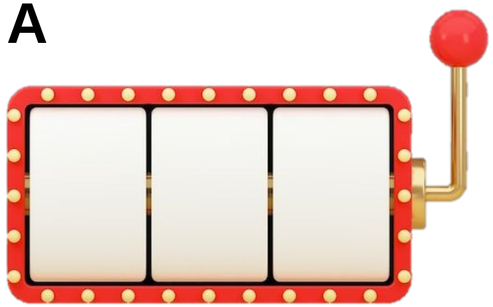
$$\pi = \frac{e^0}{e^0+e^5+e^0+e^0} = 0.66\%$$

The Multi-Armed Bandit

Softmax Action Preference

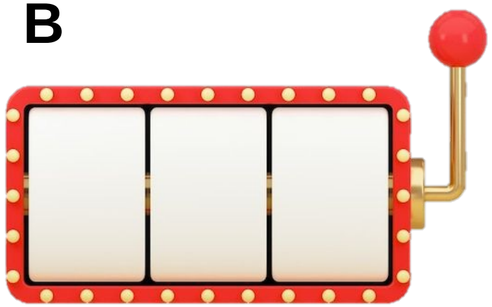
$$\pi(a|s) = e^{Q(s,a)/\tau} / \sum_b e^{Q(s,b)/\tau}$$

A



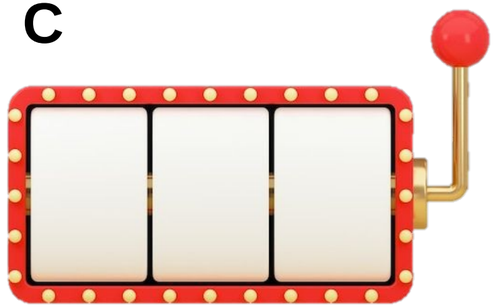
-3

B

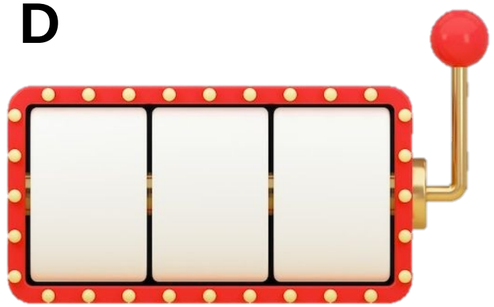


+5

C



D



$$\pi = \frac{e^{-3}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 0.03\%$$

$$\pi = \frac{e^{5}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 98.67\%$$

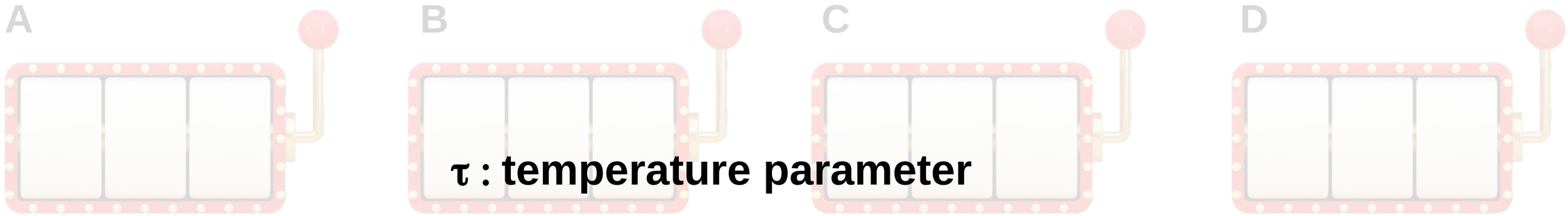
$$\pi = \frac{e^{0}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 0.66\%$$

$$\pi = \frac{e^{0}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 0.66\%$$

The Multi-Armed Bandit

Softmax Action Preference

$$\pi(a|s) = e^{Q(s,a)/\tau} / \sum_b e^{Q(s,b)/\tau}$$



τ : temperature parameter

-3

+5

Larger **τ** : more random

Smaller **τ** : more greedy

$$\pi = \frac{e^{-3}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 0.03\%$$

$$\pi = \frac{e^{5}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 98.67\%$$

$$\pi = \frac{e^{0}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 0.66\%$$

$$\pi = \frac{e^{0}}{e^{-3}+e^{5}+e^{0}+e^{0}} = 0.66\%$$



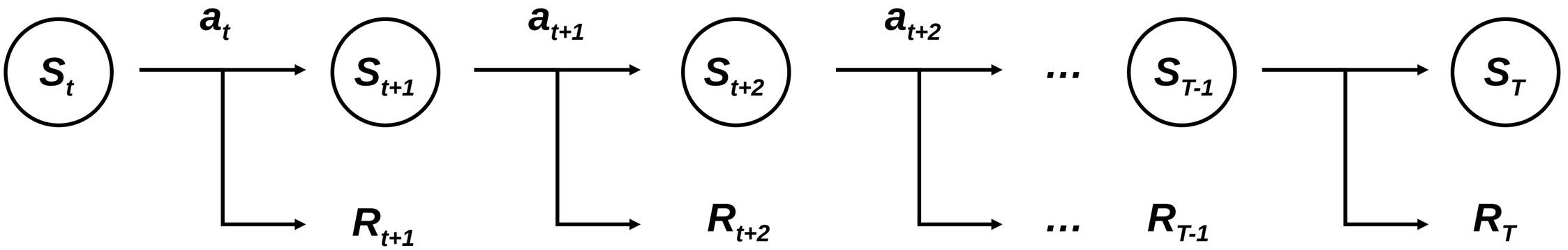
PART 2

Scaling up — Deep RL

From small look-up tables to neural networks that handle the messy, high-dimensional real world.

2

The Q-Function : What is difference between Q and V?



$$G_t = \gamma \times R_{t+1} + \gamma^1 \times R_{t+2} + \gamma^2 \times R_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$
$$= \sum_a \pi(s|a) \sum_{s',r} p(s', r|s, a)[r + \gamma V_\pi(s')]$$

Expected return when in state “s”

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Expected return when acting “a” in state “s”

REINFORCEMENT LEARNING · PART 2 · DEEP RL

The Q-Function : What is difference between Q and V?

Value rates a state. Q rates an action.

Q(state, action) answers: "if I do THIS here, how good will things turn out?" Compute it for every action, then simply pick the highest — that greedy choice is your policy.

The learning idea: nudge each estimate toward reward + best next Q. No model of the world needed — learn straight from experience.

	V
S1	0.425
S2	0.5
S3	0.525

	Q			
	↑ (A1)	↓ (A2)	← (A3)	→ (A3)
S1	0.2	0.5	0.1	**0.9**
S2	**0.6**	0.3	0.7	0.4
S3	**0.8**	0.2	0.5	0.6

The Q-Function

Off-Policy Q-Learning Update

The tabular temporal-difference rule that converges to the optimal action value q^*:

Unlike Sarsa, Q-learning bootstraps off the best next greedy action, which is what makes it off-policy.

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$$

Target for optimal Q value using the Bellman equation

Current Q value

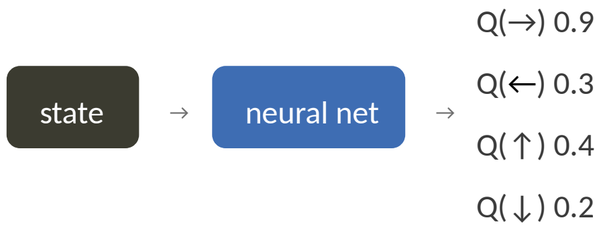
REINFORCEMENT LEARNING · PART 2 · DEEP RL

Deep RL: When the Table Gets Too Big

The problem: a real world has astronomically many states — every frame of a video game is different. No table can list them all.

The idea: replace the look-up table with a **neural network** $\hat{v}(s,w)$ that reads the state and outputs Q-values, and generalises to states it has never seen.

DQN learned to play Atari games at human level from raw pixels, using two stabilising tricks: **experience replay** and a **target network**.



Instead of listing every elements of each state – action pair,

Just get the Actions estimated by the neural net based only on the State!

Updating rule of Deep RL

Target y and Loss L is the same as Q function. The only difference is the updating rule - w rather than Q!

$$\text{Target } y = r + \gamma \max_a Q(s', a; w)$$
$$\text{Loss } L(w) = [y - Q(S, A; w)]^2$$

$$w_{t+1} \leftarrow w_t + \alpha \nabla_w L(w)$$

Objective functions minimise state-distribution weighted error to handle the scaling constraint.

Deep RL is much harder to use

Most of the challenge in DeepRL stems from the Neural net.

1. Non-i.i.d. Data (Highly Correlated Samples)

The Problem: Sequential agent experiences create strongly correlated data, breaking the i.i.d. assumption.

Why it fails: The network overfits to recent trajectories and suffers from catastrophic forgetting.

DQN Solution: Replay Buffer randomizes sample batches to break temporal correlation.

2. Moving Target Problem (Non-stationary Targets)

The Problem: The TD-target continuously shifts because it is updated by the same network being trained.

Why it fails: Chasing a moving goalpost causes severe oscillations and training divergence.

DQN Solution: Target Network freezes target parameters $\widehat{w}$ to provide a stable learning reference.

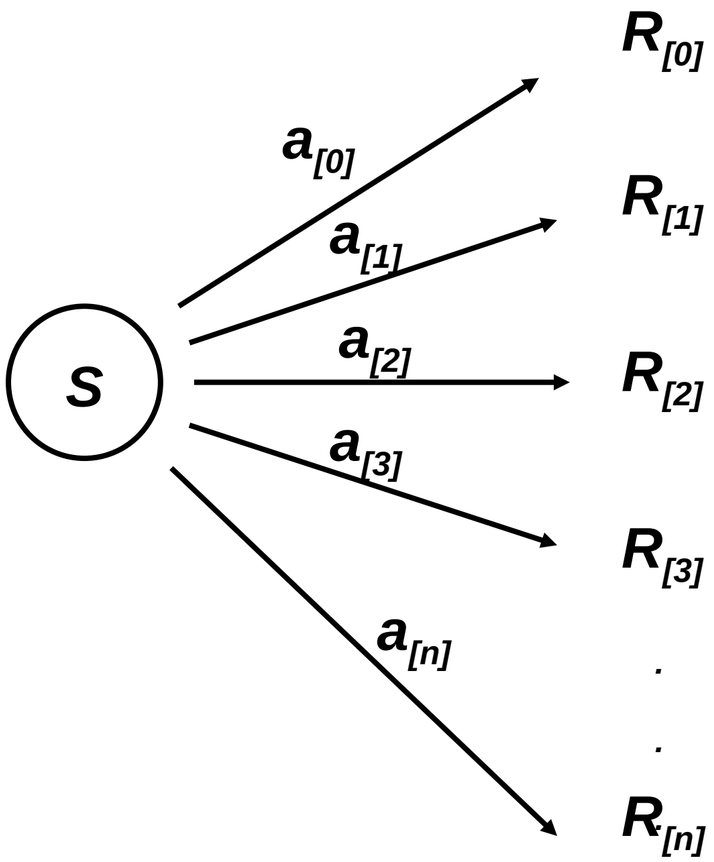
3. Policy-Data Interdependency (Shifting Distributions)

The Problem: As the policy improves, the agent explores new paths, constantly shifting the training data distribution.

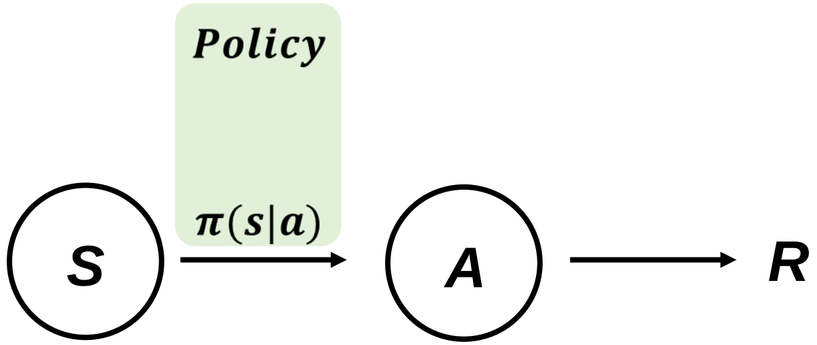
The Core Nature: The model's own behavior alters its "textbook," creating a feedback loop that hinders stable optimization.

Actor–Critic

Q-based learning



Policy-based learning



REINFORCEMENT LEARNING • PART 2 • DEEP RL

Actor–Critic

Two collaborators: one acts, the other judges

The Actor — chooses actions, the policy. Learns "what to do."

The Critic — estimates value, guiding the actor with "better / worse than expected."

Best of both worlds: the smooth action-picking of policy methods with the stable feedback of value methods.



Actor acts, Critic judges — together they learn faster and more stably than either alone.

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}; w) - V(S_t; w)$$

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla ln\pi(A_t \mid S_t; \theta)$$

$$w \leftarrow w + \alpha_w \delta_t \nabla V(S_t; \theta)$$

Actor–Critic : How to update

Target for optimal V value using the Bellman equation

Current V value

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}; w) - V(S_t; w)$$

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla ln\pi(A_t \mid S_t; \theta)$$

$$J(\theta) = \mathbb{E}_\pi[G_t]$$

$$\nabla J(\theta) = \mathbb{E}_\pi[\sum_a q_\pi(s,a)\nabla\pi(a|s;\theta)]$$

$$\nabla J(\theta) = \mathbb{E}_\pi[\sum_a q_\pi(s,a)\pi(a|s;\theta) \cdot \frac{\nabla\pi(a|s;\theta)}{\pi(a|s;\theta)}]$$

$$\approx R_{t+1} + \gamma V(S_{t+1}) \qquad \ln\pi(A_t|S_t)$$

$$\nabla J(\theta) \propto \delta_t \nabla ln\pi(A_t|S_t)$$

$$w \leftarrow w + \alpha_w \delta_t \nabla V(S_t; w)$$

$$L(w) = \frac{1}{2}[\delta_t]^2$$

$$\nabla L(w) = \delta_t \cdot \nabla(R_{t+1} + \gamma V(S_{t+1}; w) - V(S_t; w))$$

Constant target
(semi-gradient)

$$= -\delta_t \cdot \nabla V(S_t; w)$$

PART 3

RL in the Brain

The very same reward-prediction logic turns out to be written into our neural circuitry.

3

Temporal-Difference Error

Q3.**TD(0)** 알고리즘은 Bootstrapping과 Monte Carlo를 결합한 방법이다.(T/F)

Bootstrapping

Use the current estimate as the targe

$$G_t = R_{t+1} + \gamma V(S_{t+1})$$

Monte Carlo

Wait until the episode ends

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

TD(0)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Bootstrapping

→ $G_t = R_{t+1} + \gamma V(S_{t+1})$

TD(1)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Bootstrapping

→ $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$

REINFORCEMENT LEARNING • PART 3 • THE BRAIN

The Dopamine Circuit

Dopamine doesn't signal reward — it signals surprise

Dopamine neurons fire in proportion to "better or worse than I expected" — the **reward prediction error**. This is exactly the **TD error** that drives learning in RL.

1. Unexpected reward → dopamine bursts.
2. A cue that predicts reward → the burst moves to the cue.
3. Expected reward is withheld → dopamine dips below baseline.

Dopamine is, essentially, the brain's TD error — reinforcement learning is not only an algorithm, it is biology.

Biological Realization of TD Error

Dopaminergic signalling mirrors the classic temporal-difference error formula:

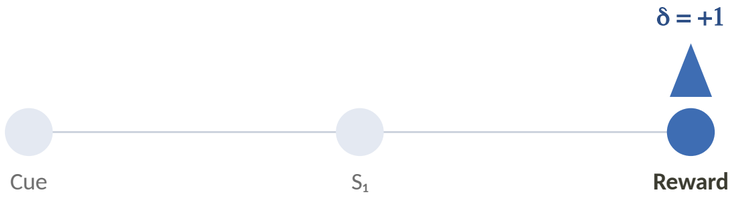
$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

This confirms biological learning systems compute the same reward-prediction dynamics as our RL algorithms.

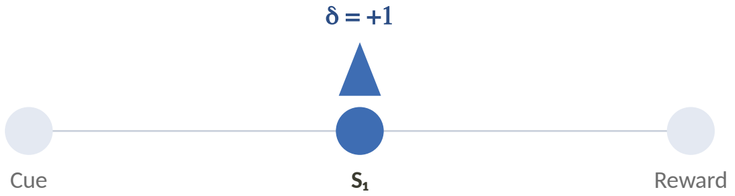
How the Burst Moves to the Cue

Early on, the error sits at the reward. Trial after trial, the TD rule pushes it one step earlier — until it lands on the cue.

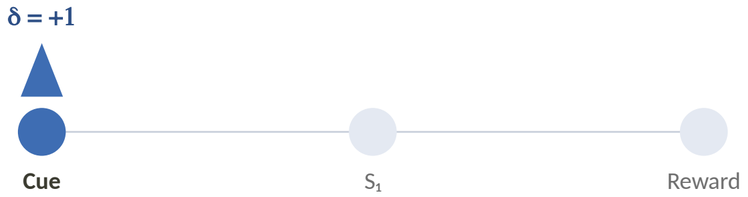
Trial 1 *error at the reward*



Trial 2 *one step earlier*



After learning *error now on the cue*



The math that moves it

Chain: cue → S_1 → reward with r = 1, V = 0, α = 1, γ = 1

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

Trial 1 · *error appears at the reward*

at reward: $\delta = r - V(S_1) = 1 - 0 = +1$

update: $V(S_1) \leftarrow 1$

Trial 2 · *the error jumps one step earlier*

at S_1: $\delta = V(S_1) - V(\text{cue}) = 1 - 0 = +1$

at reward: $\delta = 1 - V(S_1) = 1 - 1 = 0$

update: $V(\text{cue}) \leftarrow 1$

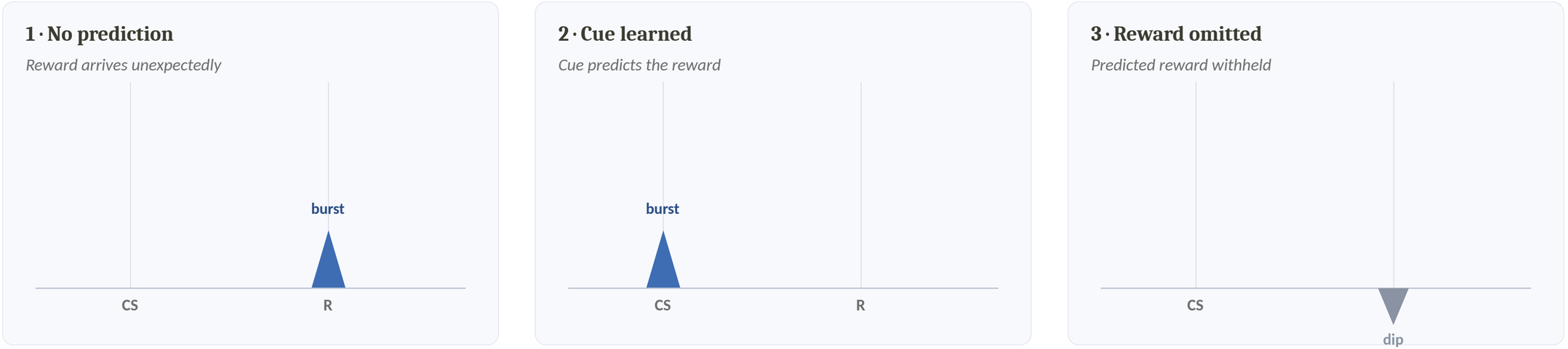
After learning · *the error now sits on the cue*

at cue: $\delta = V(\text{cue}) - 0 = +1$

at reward: fully predicted, $\delta = 0$

Recorded in the Brain

Schultz recorded midbrain dopamine neurons in monkeys given juice. The firing pattern tracked the TD prediction error exactly.



Read left to right: the burst is not glued to the reward — it migrates to the **earliest reliable predictor**, and a missing reward carves a **dip below baseline**. Exactly what $\delta = r + \gamma V(s') - V(s)$ predicts.

Key papers

Schultz, W., Dayan, P., & Montague, P. R. (1997). A neural substrate of prediction and reward. **Science, 275**(5306), 1593–1599. · Schultz, W. (1998). Predictive reward signal of dopamine neurons. **J. Neurophysiol., 80**(1), 1–27.

REINFORCEMENT LEARNING • PART 3 • THE BRAIN

Model-Free vs. Model-Based

One question separates them: does the agent build a model of T — a map of where its actions lead?

Experience: (s, a, r, s′)

MODEL-FREE

Skips the model entirely

(s, a, r, s′) → update Q(s,a)

Learns **how good** each action turned out — directly from what happened. It **never represents where actions lead.**

builds a model of T? ✖ No

MODEL-BASED

Learns T, then plans with it

(s, a, r, s′) → estimate T(s′|s,a)

→ plan → values

First learns the **map of where actions lead**, then reasons over that map to value each action.

builds a model of T? ✔ Yes

REINFORCEMENT LEARNING • PART 3 • THE BRAIN

Spot the Difference: Is T in the Equation?

Same goal — value each action. Model-free never touches T; model-based's value is built out of it.

MODEL-FREE

Stores just the action values

$Q(s,a)$ — *one cached number per action*

Uses the TD update

$$Q(s,a) \leftarrow Q(s,a) + \alpha[\, r + \gamma \max Q(s') - Q(s,a)\,]$$

No T anywhere. It just takes the **one real s′** the world happened to give, and learns from that single outcome.

MODEL-BASED

Stores the model itself

$T(s'|s,a)$ and $R(s,a)$ — *where actions lead*

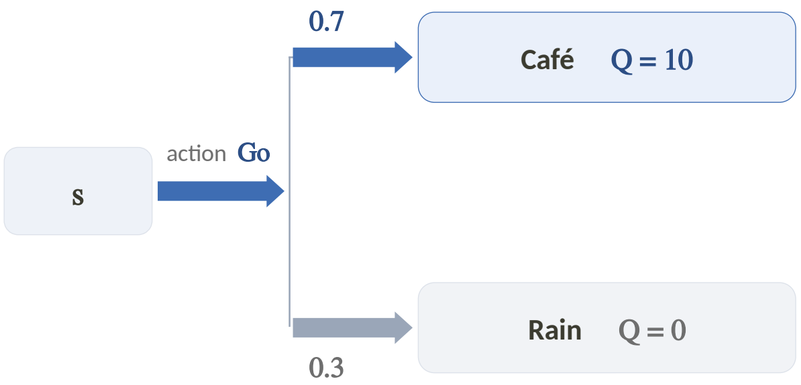
Uses the Bellman backup

$$Q(s,a) = R(s,a) + \gamma \sum_{s'} T(s'|s,a) \max Q(s')$$

That Σ over T is the model — it averages over **every place** the action could lead, without going there.

Model-Based in Action

The stored model says where an action leads — as probabilities. Planning just plugs them in. One step, no real trip.



Same action, two possible outcomes. The model stores each branch and its probability.

The stored model

$T(\text{Café}|s,\text{Go})=0.7 \quad T(\text{Rain}|s,\text{Go})=0.3 \quad R(s,\text{Go})=0$

Plan one step (Bellman)

$$Q(s,\text{Go}) = R + \gamma \sum_{s'} T(s'|s,\text{Go})\, Q(s')$$

$= 0 + [\, 0.7 \times 10 \;+\; 0.3 \times 0 \,]$

$= 7$

The 0.3 bad branch pulls the value from 10 down to 7 — and the agent knew that **without ever getting rained on.**

REINFORCEMENT LEARNING • PART 3 • THE BRAIN

When the World Changes

Same Café/Rain world as before. Now the Café is devalued — and the two systems part ways.

The Café is devalued: its reward drops $R(\text{Café})\ 10 \to 2$ (you're now full). The agent learns this from **one** taste — but hasn't re-walked Go from s.

MODEL-BASED *edits its learned model*

Change one entry it already stores:

$R(\text{Café})\ 10 \to 2$

Re-plan from s — recompute:

$$Q(s,\text{Go}) = 0.7\times2 + 0.3\times0 = 1.4$$

✓ **Recomputed from the model — no new trip.**

MODEL-FREE *cache is now stale*

Its cached value is untouched:

$Q(s,\text{Go}) = 7$ *still stale*

No T to edit — must go again and again:

$$Q(s,\text{Go}):\ 7 \to 4.5 \to \ldots \to 1.4 \quad \textit{(many trips)}$$

✗ **Stuck on the old value until re-experienced.**

Why this is **learning**, not just planning: the 0.7 / 0.3 and the rewards were **estimated from real visits**. If T were handed to us in full, this would be planning a known MDP — not model-based **RL**.

REINFORCEMENT LEARNING · WRAP-UP

The through-line

One idea — learn from reward — carried us from a slot machine, through the Bellman equation and Q-learning, to the dopamine system.

1 · Foundations

Agents learn by trial and error to maximise long-run value via the Bellman equation; explore vs. exploit is the core tension.

2 · Deep RL

Q-values compare actions; neural networks scale this to huge worlds; Actor–Critic makes the policy-gradient stable.

3 · The Brain

Dopamine encodes the same TD prediction error; habit (model-free) and planning (model-based) work together.

Big picture: reward-driven learning is a bridge between machine intelligence and the biology of the brain.

Thank You

Questions, intuitions, and counter-examples all welcome.

Reinforcement Learning · From trial and error to the brain